EcommerceMultiVendor — Module Deep Dive

Auth module

• **Responsibility**: register/login, JWT issuance, role assignment.

• **Packages**: controller.AuthController, service.AuthService, security.*, dto.auth.*.

• **Endpoints**: POST /api/v1/auth/register, POST /api/v1/auth/login.

• **Entities**: User (CUSTOMER | SELLER | ADMIN).

• **Flow**: validate DTO → encode password / authenticate → persist or load user → JwtService.generateToken → AuthResponse.

• **Config**: app.jwt.secret, app.jwt.expiration-ms.

Catalog module

• **Responsibility**: public product/category browse, reviews.

• **Packages**: CatalogService, ProductController, Product / Category / Review.

• **Endpoints**: GET /products, GET /products/{id}, GET /categories, reviews GET/POST.

• **Flow**: optional q / categoryId filters → map to ProductResponse.

• **Notes**: write APIs for products live under Seller; category create under Admin.

Cart & wishlist

• **Responsibility**: per-user cart quantities and wishlist membership.

• **Endpoints**: /api/v1/cart, /api/v1/cart/items, /api/v1/wishlist/{productId}.

• **Flow**: get-or-create cart → upsert CartItem with stock checks → totals in CartResponse.

• **Wishlist**: unique (user, product); delete uses @Modifying repository method.

Order & address

• **Responsibility**: shipping addresses, checkout, order history, cancel, seller status updates.

• **Endpoints**: /addresses, /checkout, /orders, /orders/{id}/cancel, seller PATCH .../status.

• **Flow**: validate cart + address → build order lines → apply coupon → reduce stock → clear cart → mail.

• **Statuses**: PENDING → PAID → SHIPPED → DELIVERED | CANCELLED | REFUNDED.

Payment module

• **Responsibility**: create/confirm payments for Razorpay, Stripe, COD.

• **Packages**: PaymentService, PaymentController, PaymentProperties.

• **Flow**: ensure order PENDING and owned by caller → call provider SDK or mock → store Payment → confirm captures and marks order PAID.

• **Config**: app.payment.razorpay.*, app.payment.stripe.*.

Seller module

• **Responsibility**: shop profile, inventory, seller orders, earnings dashboard.

• **Endpoints**: /api/v1/seller/shop, /products, /orders, /dashboard.

• **Flow**: create shop (pending approval) → after admin approval create products → fulfill orders → chart series from order dates.

• **Guard**: unapproved shops cannot publish products.

Admin module

• **Responsibility**: approve shops, manage categories/coupons, platform metrics.

• **Endpoints**: /api/v1/admin/dashboard, /shops/pending, /shops/{id}/approve, /categories, /coupons.

• **Flow**: list pending shops → approve/reject + email seller → seed coupon codes such as WELCOME10.

Chatbot module

• **Responsibility**: rule-based assistant over domain services.

• **Packages**: chatbot.IntentMatcher, chatbot.ChatIntent, ChatbotService, ChatController.

• **Endpoint**: POST /api/v1/chat with { message } → { intent, reply }.

• **Intents**: GREETING, HELP, FAQ_*, LIST_ORDERS, ORDER_DETAIL/STATUS, CANCEL_ORDER, SHOW_CART, PRODUCT_SEARCH, PRODUCT_BY_CATEGORY, SHOW_WISHLIST, seller/admin summaries, UNKNOWN.

• **Flow**: match → authorize role when needed → call service → format short text reply.

• **UI**: floating ChatWidget on authenticated pages.

Frontend module

• **Responsibility**: role-aware SPA (storefront, cart/checkout, seller/admin dashboards).

• **State**: Redux authSlice, cartSlice; Axios interceptor attaches JWT from localStorage.

• **Routing**: React Router under Layout; Private guard for authenticated routes.

• **Forms**: Formik + Yup on login/register; MUI controls elsewhere; Recharts on dashboards.

• **Build**: Vite + Tailwind v4 plugin; production nginx proxies /api to backend.

Cross-cutting

• **Exceptions**: ApiException + GlobalExceptionHandler → consistent JSON errors.
• **OpenAPI**: springdoc UI at /swagger-ui.html with bearer scheme.
• **Seeding**: DataSeeder creates demo users, categories, approved shop, products, coupon when DB empty.
• **Mail**: MailNotificationService logs failures instead of failing business transactions.